

Master Implementation Blueprint

Project: Hybrid Explainable Heterogeneous Graph Anomaly Detection for Healthcare Provider Fraud Detection

Role: Lead ML Engineer / Research Scientist specification

Status: Implementation-ready — research planning is closed

Implementer: 1 engineer | **Reviewers:** 2 (validation + paper)

Executive Summary

This blueprint defines a **production-grade research repository** (hgad-cms-fraud) that implements:

1. CMS data → heterogeneous graph construction
2. **Primary:** R-GCN + Isolation Forest + α -fusion + GNNExplainer
3. **Fallback:** GraphSAGE + same downstream stack
4. Six baselines under provider-disjoint 5-fold CV
5. Ablation suite, statistical tests, publication artifacts

Design principles: config-driven, fold-safe (no leakage), reproducible seeds, artifact versioning, 4GB-GPU-safe coarsening.

1. Repository Structure

hgad-cms-fraud/

```
├— README.md
├— pyproject.toml
├— requirements.txt
├— requirements-dev.txt
├— .gitignore
├— .env.example
├— Makefile
├— configs/
├— data/
|   ├── raw/           # gitignored — user places Kaggle CSVs
|   ├── processed/
|   └— splits/
├— artifacts/
```

```

|   ├── graphs/
|   ├── models/
|   ├── embeddings/
|   ├── explanations/
|   └── results/
├── notebooks/      # optional EDA only; not in critical path
├── scripts/
├── src/
|   └── hgad_cms/
├── tests/
├── docs/
|   ├── protocol.md    # frozen experimental protocol
|   ├── graph_schema.md
|   └── reproduction.md
└── paper/
    ├── tables/
    └── figures/

```

2. Folder Structure (Detailed)

Path	Purpose	Git tracked
data/raw/	Original Kaggle CSVs (unchanged filenames)	X
data/processed/	Merged claims, entity tables, feature matrices	X (manifest only)
data/splits/	Fold provider ID lists JSON	✓
artifacts/graphs/	Per-fold HeteroData .pt files	X
artifacts/models/	Checkpoints {fold,model,ablation}.pt	X
artifacts/embeddings/	Provider embeddings per fold	X
artifacts/explanations/	Subgraph masks, fidelity JSON	X

Path	Purpose	Git tracked
artifacts/results/	Metrics CSV, significance tests	✓ (summary only)
configs/	YAML experiment configs	✓
src/hgad_cms/	Package source	✓
tests/	Unit + integration tests	✓
scripts/	CLI entry points	✓
paper/figures/	Generated publication plots	✓

3. Python Package Structure

src/hgad_cms/

```

├── __init__.py      # __version__ = "0.1.0"
├── cli.py           # Typer/argparse main entry
├── constants.py     # Column names, node/edge type strings
├── exceptions.py
|
├── config/
|   ├── __init__.py
|   ├── schema.py    # Pydantic/dataclass config models
|   └── loader.py     # load_config(path) -> Config
|
├── data/
|   ├── __init__.py
|   ├── loader.py    # load_raw_cms(data_dir)
|   ├── merger.py    # merge claims + labels
|   ├── cleaner.py   # dtypes, dates, gender normalize
|   ├── validator.py # schema + row count checks
|   └── splits.py     # stratified provider K-fold
|

```

```

└─ features/
|   └─ __init__.py
|   └─ provider_aggregates.py # tabular provider features
|   └─ entity_features.py    # bene/phy/dx node features
|   └─ centrality.py        # RF+centrality baseline features
|   └─ vocabulary.py        # top-K dx/proc vocab (fold-safe)
|   └─ normalizer.py        # fold-fit scalers
|
└─ graph/
|   └─ __init__.py
|   └─ schema.py            # node/edge type definitions
|   └─ builder.py           # build_hetero_graph(...)
|   └─ coarsening.py        # top-N bene/phy, UNK bucket
|   └─ projection.py        # homo projection for GraphSAGE
|   └─ masks.py            # train/val/test provider masks
|   └─ io.py                # save/load HeteroData
|
└─ models/
|   └─ __init__.py
|   └─ rgcn.py              # HybridRGCNEncoder
|   └─ graphsage.py         # HybridSAGEEncoder (fallback)
|   └─ heads.py             # ProviderClassifierHead
|   └─ fusion.py            # ScoreFusionModule
|   └─ factory.py           # build_model(config)
|
└─ anomaly/
|   └─ __init__.py
|   └─ isolation_forest.py  # fit/score on embeddings
|   └─ normalization.py    # min-max on val scores
|

```

```
└─ baselines/
|   └─ __init__.py
|   └─ sklearn_models.py    # LR, RF
|   └─ catboost_model.py    # CatBoost wrapper
|   └─ trainer.py           # unified baseline CV loop
|
└─ training/
|   └─ __init__.py
|   └─ trainer.py           # RGCN/SAGE trainer
|   └─ loss.py              # weighted BCE, focal
|   └─ sampler.py           # NeighborLoader factory
|   └─ early_stopping.py
|   └─ checkpoint.py
|
└─ evaluation/
|   └─ __init__.py
|   └─ metrics.py           # AUPRC, F1, Recall@K, etc.
|   └─ cross_validation.py  # outer fold orchestrator
|   └─ ablation.py          # ablation registry + runner
|   └─ statistics.py        # paired t-test, Holm, Cohen's d
|
└─ explain/
|   └─ __init__.py
|   └─ ego_subgraph.py      # k-hop extraction
|   └─ gnn_explainer.py    # wrapper + fidelity
|   └─ motif_coder.py       # M1-M5 motif tagging
|   └─ export.py            # JSON/GraphML case studies
|
└─ tracking/
|   └─ __init__.py
```

```
| ├── logger.py      # structlog/logging setup
| ├── experiment.py  # run_id, metadata JSON
| └── mlflow_optional.py # optional MLflow adapter
|
└── utils/
    ├── __init__.py
    ├── seed.py      # set_global_seed
    ├── device.py    # get_device, memory stats
    ├── paths.py     # artifact path helpers
    └── timing.py    # context manager for wall clock
```

4. Exact Module Dependencies

requirements.txt (pinned ranges)

Core

python>=3.10,<3.13

numpy>=1.24,<2.0

pandas>=2.0,<3.0

pyarrow>=14.0

scipy>=1.11

scikit-learn>=1.3,<1.6

catboost>=1.2,<2.0

Deep learning

torch>=2.1,<2.4

torch-geometric>=2.4,<2.6

Install PyG wheels matching torch+CUDA via documented index URL

Config & CLI

pyyaml>=6.0

pydantic>=2.0

typer>=0.9

rich>=13.0

Metrics & stats

torchmetrics>=1.0

Explainability (PyG built-in GNNExplainer)

IO & tracking

joblib>=1.3

jsonschema>=4.0

Plotting (results only)

matplotlib>=3.7

seaborn>=0.13

requirements-dev.txt

pytest>=7.4

pytest-cov>=4.1

hypothesis>=6.0

ruff>=0.1

mypy>=1.7

pre-commit>=3.5

External system dependencies

- CUDA 11.8 or 12.1 (match PyTorch wheel)
- Git LFS optional for large checkpoints
- ~10 GB disk for processed data + artifacts

5. Data Preprocessing Pipeline

Module: hgad_cms.data

5.1 loader.py

Purpose	Load raw CSVs with canonical column mapping
Inputs	data/raw/ path from config
Outputs	RawCMSData dataclass: providers, beneficiaries, inpatient, outpatient
Functions	load_raw_cms(data_dir: Path) -> RawCMSData detect_file_names(data_dir) -> dict[str, Path]
Dependencies	pandas, constants.py

Unit tests	test_loader_finds_files, test_loader_row_counts, test_loader_columns
-------------------	--

Canonical filenames (glob fallback):

constants.py

PROVIDER_FILE = "Train-*.csv"

BENEFICIARY_FILE = "Train_Beneficiarydata-*.csv"

INPATIENT_FILE = "Train_Inpatientdata-*.csv"

OUTPATIENT_FILE = "Train_Outpatientdata-*.csv"

5.2 merger.py

Purpose	Concat IP+OP claims; join provider labels
Inputs	RawCMSData
Outputs	claims: DataFrame, providers: DataFrame, beneficiaries: DataFrame
Functions	merge_claims(ip, op) -> DataFrame attach_claim_type(claims) -> DataFrame # is_inpatient: bool merge_provider_labels(claims, providers) -> DataFrame
Dependencies	pandas
Unit tests	test_claim_count_558211, test_all_providers_in_labels, test_no_duplicate_claim_ids

5.3 cleaner.py

Purpose	Normalize types and known data quality issues
Inputs	Merged DataFrames
Outputs	Clean DataFrames
Functions	normalize_gender(series) -> Series # map {F,M,m,f,FF,MF} → {0,1,UNK} parse_dates(series) -> Series # mixed format, dayfirst=False then mixed fill_physician_nulls(claims) -> DataFrame clip_reimbursement_outliers(claims, q=0.999) -> DataFrame
Dependencies	pandas
Unit tests	test_gender_mapping, test_date_parse_rate, test_reimbursement_non_negative

5.4 validator.py

Purpose	Hard-stop validation before graph build
Inputs	Clean tables
Outputs	ValidationReport (pass/fail + stats)
Functions	validate_cms_data(claims, providers, beneficiaries) -> ValidationReport
Checks	5410 providers; fraud rate 0.09 ± 0.01 ; claim-provider join 100%; beneficiary join $\geq 99\%$
Unit tests	test_validator_passes_on_real_sample (use 100-row fixture)

5.5 Pipeline orchestrator: scripts/01_preprocess.py

python scripts/01_preprocess.py --config configs/default.yaml

Outputs:

data/processed/claims.parquet

data/processed/providers.parquet

data/processed/beneficiaries.parquet

data/processed/manifest.json # row counts, hash, timestamp

6. Graph Construction Pipeline

Module: hgad_cms.graph

Orchestrator: scripts/02_build_graphs.py

Runs **per fold** after splits exist.

artifacts/graphs/fold_{k}/hetero_data.pt

artifacts/graphs/fold_{k}/graph_stats.json

6.1 coarsening.py

Purpose	Cap node counts for 4GB GPU
Inputs	claims (outer-train scoped), config caps
Outputs	entity_maps: CoarsenMaps (raw_id $\rightarrow$ node_idx, UNK idx)
Functions	build_vocabulary(ids, top_k) -> dict map_to_vocab(id, vocab) -> int

Defaults	max_beneficiaries=20000, max_physicians=20000, max_diagnoses=500
Unit tests	test_unk_bucket_exists, test_all_providers_retained

6.2 builder.py

Purpose	Construct PyG HeteroData
Inputs	Processed tables, fold split, coarsen maps, fitted normalizers
Outputs	HeteroData with features, edges, provider labels, masks
Functions	build_hetero_graph(...) -> HeteroData _build_edges_treated(...) -> EdgeIndex _build_edges_affiliated(...) -> EdgeIndex _build_edges_diagnosed(...) -> EdgeIndex _build_edges_shares_patient(...) -> EdgeIndex (optional ablation flag)
Dependencies	torch, torch_geometric, features.*
Unit tests	test_hetero_node_counts, test_edge_directions, test_provider_label_alignment, test_no_provider_features_from_test_claims (leakage)

Edge construction rules:

Edge	Construction	Weight attr
("beneficiary", "treated", "provider")	GroupBy (bene, provider) on claims	[claim_count, total_reimb] log1p
("physician", "affiliated_with", "provider")	AttendingPhysician non-null	[co_claim_count]
("provider", "diagnosed", "diagnosis")	Explode CImDiagnosisCode_1..10	[frequency]
("provider", "shares_patient", "provider")	Pairs sharing ≥ 2 beneficiaries	[shared_count, jaccard]

6.3 masks.py

Purpose	Provider node masks for inner-train/val and outer-test
Functions	assign_provider_masks(data, split_dict) -> HeteroData
Outputs	data["provider"].train_mask, .val_mask, .test_mask (bool tensors)

Unit tests	test_masks_disjoint, test_mask_fraud_rate_preserved
-------------------	---

7. Feature Engineering Pipeline

Module: hgad_cms.features

7.1 provider_aggregates.py (tabular baselines + optional provider node features)

Purpose	Provider-level tabular matrix $\mathbf{X} \in \mathbb{R}^{5410 \times F}$
Inputs	claims scoped to outer-train providers only for fit; transform all
Outputs	provider_features: np.ndarray, feature_names: list[str]
Functions	fit_provider_aggregates(claims_train, config) -> AggregatorState transform_provider_aggregates(claims, state) -> np.ndarray
Feature list (F≈120)	n_claims, n_patients, n_physicians, total/mean/std reimb, ip_ratio, mean duration, mean deductible, top-50 dx counts, top-50 proc counts, bene mean age, bene gender ratio, mean chronic count
Unit tests	test_aggregate_no_nan, test_fold_fit_transform_no_leakage

7.2 entity_features.py (graph node features)

Node type	Features (dim)
provider	Tabular aggregates (F) + degree stats (4)
beneficiary	age, gender, state_id, chronic_sum, annual_reimb (5 + optional)
physician	claim_count, n_providers, n_patients (3)
diagnosis	frequency, log_frequency (2)
Functions	build_node_features(data_tables, maps, state) -> dict[str, Tensor]
Unit tests	test_feature_dims_match_schema, test_diagnosis_only_train_vocab

7.3 centrality.py (B4 baseline)

Purpose	Compute centrality on provider projection graph
Functions	compute_provider_centralities(edge_index, n_providers) -> np.ndarray

Metrics	degree, betweenness (approx sampled), closeness (approx), eigenvector
Dependencies	networkx or igraph (CPU)
Unit tests	test_centrality_shape, test_isolated_provider_handled

7.4 normalizer.py

Functions	FoldNormalizer.fit(X_train), .transform(X) — StandardScaler per fold
Unit tests	test_normalizer_fit_only_train

8. Graph Schema Implementation Details

hgd_cms/graph/schema.py

NODE_TYPES = ("provider", "beneficiary", "physician", "diagnosis")

EDGE_TYPES = (

 ("beneficiary", "treated", "provider"),

 ("physician", "affiliated_with", "provider"),

 ("provider", "diagnosed", "diagnosis"),

 ("provider", "shares_patient", "provider"), # *ablation-toggle*

)

TARGET_NODE = "provider"

TARGET_ATTR = "y" *# int64 {0,1}*

TARGET_PROB = "y_prob" *# float fused score*

HeteroData layout (per fold)

data["provider"].x *# [N_p, F_p]*

data["provider"].y *# [N_p]*

data["provider"].train_mask *# [N_p] bool*

data["provider"].val_mask

data["provider"].test_mask

data["provider"].provider_id *# string list for traceability*

data[edge_type].edge_index *# [2, E]*

data[edge_type].edge_attr *# [E, d_e] optional*

Homogeneous projection (GraphSAGE fallback)

projection.py

def to_homogeneous_provider_graph(data: HeteroData) -> Data:

"""

Collapse to provider nodes only.

Edge weight = aggregated multi-hop proxy OR

use shares_patient + diagnosed Jaccard similarity.

Node features = provider.x (already rich).

"""

9. R-GCN Implementation Plan

Module: `hgad_cms/models/rgcn.py`

Purpose	Primary encoder: 2-layer heterogeneous R-GCN
Inputs	HeteroData, config
Outputs	Provider embeddings Z_p [N_p , d]; logits [N_p , 1]

Classes

class RGCNEncoder(nn.Module):

def __init__(self, metadata, in_channels_dict, hidden_dim, num_layers,
num_bases, dropout, edge_types): ...

def forward(self, x_dict, edge_index_dict) -> dict[str, Tensor]: ...

class HybridRGCNModel(nn.Module):

encoder: RGCNEncoder

head: ProviderClassifierHead

def forward(self, batch) -> tuple[Tensor, Tensor]: *# logits, z_provider*

def encode(self, data) -> Tensor: *# inference-only provider embeddings*

Implementation notes

- Use `torch_geometric.nn.HeteroConv` wrapping `RGCNConv` per edge type
- `num_bases=4` (relation decomposition — memory efficient)
- `hidden_dim=64`, `num_layers=2`, `dropout=0.3`
- Apply `LayerNorm` on provider embeddings before head
- **Loss:** `weighted_bce_with_logits(logits[train_mask], y[train_mask], pos_weight)`

Training (hgas_cms/training/trainer.py)

class RGCNTrainer:

def train_fold(self, data, config, fold_id) -> TrainResult:

NeighborLoader on provider seed nodes

batch_size=512, num_neighbors=[15,10]

Adam lr=1e-3, weight_decay=5e-4

Early stopping on val AUPRC, patience=15

AMP: torch.cuda.amp.autocast enabled

| **Unit tests** | test_rgc_forward_shapes, test_rgc_grad_flow, test_encode_deterministic_eval |

| **Integration** | test_rgc_one_epoch_smoke on 50-provider synthetic heterograph |

10. GraphSAGE Implementation Plan

Module: hgas_cms/models/graphsage.py

Purpose	Fallback encoder if R-GCN OOM or fails convergence by Phase 5 gate
Inputs	Homogeneous Data (provider projection)
Outputs	Same interface as R-GCN: encode(), logits

Classes

class GraphSAGEEncoder(nn.Module):

2 x SAGEConv(hidden=64, aggregator=mean)

class HybridGraphSAGEModel(nn.Module):

Identical head interface to HybridRGCNModel

Gate criteria (Phase 5 → fallback)

- GPU OOM after coarsening + AMP → switch to GraphSAGE
- Val AUPRC < 0.55 after 50 epochs → switch (configurable)

| **Unit tests** | test_sage_forward_shapes, test_sage_matches_rgc_interface |

11. Isolation Forest Integration

Module: hgas_cms/anomaly/isolation_forest.py

Purpose	Unsupervised anomaly score on provider embeddings
Inputs	Z_train_non_fraud: np.ndarray [N_neg, d], Z_all: np.ndarray [N_p, d]
Outputs	anomaly_scores: np.ndarray [N_p] (higher = more anomalous)

Classes

@dataclass

class IFResult:

raw_scores: np.ndarray

normalized_scores: np.ndarray *# min-max fit on val non-test*

class ProviderIsolationForest:

def fit(self, z_non_fraud: np.ndarray, config: IFConfig) -> None

def score(self, z: np.ndarray) -> np.ndarray

def save(self, path: Path) -> None

@classmethod

def load(cls, path: Path) -> "ProviderIsolationForest"

Rules

- Fit **only** on inner-train **non-fraud** provider embeddings
- contamination=0.0935, n_estimators=200, random_state=42
- Use -score_samples() then invert so higher = more anomalous
- Normalize using min-max from **validation** provider scores

| **Unit tests** | test_if_fit_non_fraud_only, test_if_scores_shape, test_if_deterministic |

12. Score Fusion Implementation

Module: hgad_cms/models/fusion.py

Purpose	Combine supervised and anomaly scores
Inputs	p_fraud: np.ndarray, s_anomaly: np.ndarray, alpha: float
Outputs	r_final: np.ndarray, optimal alpha

Classes

class ScoreFusionModule:

```

def tune_alpha(
    self,
    p_val: np.ndarray,
    s_val: np.ndarray,
    y_val: np.ndarray,
    grid: np.ndarray = np.arange(0.1, 1.0, 0.1),
) -> float:
    """Select alpha maximizing val AUPRC."""

def fuse(self, p: np.ndarray, s: np.ndarray, alpha: float) -> np.ndarray:
    return alpha * p + (1.0 - alpha) * s

```

| Unit

tests | test_alpha_grid_search, test_fusion_bounds, test_alpha_0_equals_p, test_alpha_1_requires_and_maly_inverted |

13. Cross-Validation Implementation

Module: hgad_cms/evaluation/cross_validation.py

Purpose	Provider-disjoint 5-fold outer CV orchestration
Inputs	Full provider list + labels, config
Outputs	CVResults per model/ablation

Classes

@dataclass

class FoldSplit:

```

    fold_id: int
    outer_test_providers: list[str]
    inner_train_providers: list[str]
    inner_val_providers: list[str]

```

class CrossValidator:

```

    def create_splits(self, providers: pd.DataFrame, n_splits=5, seed=42) -> list[FoldSplit]
    def save_splits(self, splits, path: Path) -> None
    def load_splits(self, path: Path) -> list[FoldSplit]

```

```
class ExperimentRunner:
```

```
    def run_model(self, model_name: str, ablation: str, fold: FoldSplit) -> FoldMetrics
```

```
    def run_all(self, config: Config) -> ResultsTable
```

Per-fold execution order

1. Build graph (fold-scoped fit for vocab/normalizer)
2. Train R-GCN → save checkpoint
3. Extract embeddings → train IF → tune α on val
4. Evaluate on outer-test providers
5. (Post-hoc) GNNExplainer on selected providers

| **Unit tests** | test_splits_stratified, test_splits_provider_disjoint, test_no_label_leakage_outer_test |

14. Baseline Implementation

Module: `hgad_cms/baselines`

ID	Class	File
B1	LogisticRegressionBaseline	sklearn_models.py
B2	RandomForestBaseline	sklearn_models.py
B3	CatBoostBaseline	catboost_model.py
B4	RFCentralityBaseline	sklearn_models.py + centrality.py
B5	RGCNOnlyModel	reuse R-GCN without IF/fusion
B6	IFOnlyModel	embeddings + IF, $\alpha=0$

Unified interface

```
class BaselineModel(Protocol):
```

```
    def fit(self, X_train, y_train, X_val, y_val) -> None
```

```
    def predict_proba(self, X_test) -> np.ndarray
```

```
    def name(self) -> str
```

Graph baselines (B5, B6) use GraphModelProtocol with `fit(data)` / `predict_proba(data)`.

`baselines/trainer.py`

```
def run_baseline_cv(baseline_name, splits, features, config) -> CVResults
```

| **Unit**

tests | test_lr_binary_output, test_catboost_handles_categorical, test_rf_centrality_feature_dim |

15. Explainability Implementation

Module: hgad_cms/explain

Purpose	GNNExplainer subgraph audit for flagged providers
Trigger	After best Hybrid model per fold

15.1 ego_subgraph.py

```
def extract_provider_ego_subgraph(  
    data: HeteroData,  
    provider_idx: int,  
    num_hops: int = 2,  
    edge_types: list,  
) -> HeteroData: ...
```

15.2 gnn_explainer.py

@dataclass

class ExplanationResult:

```
    provider_id: str  
    edge_mask: dict[tuple, Tensor]  
    fidelity: float  
    sparsity: float  
    top_edges: list[dict]
```

class GNNExplainerWrapper:

```
    def explain(self, model, data, provider_idx, epochs=200) -> ExplanationResult  
    def compute_fidelity(self, model, data, mask) -> float
```

Implementation detail: Project hetero subgraph to homogeneous provider-centered graph for PyG Explainer if hetero explainer unstable; document in explain/export.py.

15.3 export.py

Outputs per provider:

artifacts/explanations/fold_{k}/{provider_id}.json

artifacts/explanations/fold_{k}/{provider_id}.graphml

| **Unit**

tests | test_ego_subgraph_node_count, test_fidelity_in_zero_one, test_explanation_json_schema |

16. Experiment Tracking

Module: hgad_cms/tracking/experiment.py

@dataclass

class ExperimentRecord:

run_id: str *# uuid4[:8]*

git_commit: str

config_hash: str

started_at: str

finished_at: str

hardware: dict

fold_metrics: list[dict]

best_alpha_per_fold: list[float]

status: str *# running | completed | failed*

Storage: artifacts/results/{run_id}/experiment.json

Optional MLflow (tracking/mlflow_optional.py): log params/metrics if MLFLOW_TRACKING_URI set; not required.

17. Logging

Module: hgad_cms/tracking/logger.py

def setup_logging(log_level: str, log_file: Path) -> logging.Logger

Format:

2026-06-04 14:32:01 | INFO | fold=2 | epoch=45 | val_auprc=0.712 | gpu_mem=3.1GB

Log files: artifacts/results/{run_id}/train.log

Log events (minimum):

- Fold start/end
- Epoch val metrics (every epoch)
- Early stop trigger
- IF fit size, α selected
- OOM warnings + batch size reductions

- Explanation fidelity summary

18. Configuration Management

configs/default.yaml (master)

project:

name: hgad-cms-fraud

seed: 42

device: cuda *# fallback cpu*

data:

raw_dir: data/raw

processed_dir: data/processed

splits_dir: data/splits

graph:

max_beneficiaries: 20000

max_physicians: 20000

max_diagnoses: 500

include_shares_patient: true

top_dx_per_claim: 10

model:

primary: rgcn *# rgcn | graphsage*

hidden_dim: 64

num_layers: 2

num_bases: 4

dropout: 0.3

lr: 0.001

weight_decay: 0.0005

epochs: 200

patience: 15

batch_size: 512

num_neighbors: [15, 10]

pos_weight: auto *# N_neg/N_pos*

anomaly:

n_estimators: 200

contamination: 0.0935

fusion:

alpha_grid: [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]

cv:

n_splits: 5

inner_val_ratio: 0.2

baselines:

enabled: [lr, rf, catboost, rf_centrality, rgcn_only, if_only]

ablations:

enabled: [full, rgcn_only, if_only, no_shares_patient, no_diagnosis, homogeneous_sage]

explain:

num_hops: 2

epochs: 200

n_tp: 30

n_fp: 10

n_tn: 10

memory:

amp: true

gradient_accumulation: 2

pin_memory: false

Config loading

config/loader.py

def load_config(path: Path, overrides: dict | None = None) -> Config

CLI overrides: --model.primary graphsage --cv.n_splits 5

| **Unit tests** | test_config_load_default, test_config_validation_rejects_bad_alpha |

19. Artifact Management

Naming convention

artifacts/models/{run_id}/fold_{k}/{model}_{ablation}.pt

artifacts/embeddings/{run_id}/fold_{k}/provider_z.npy

artifacts/graphs/fold_{k}/hetero_data.pt

artifacts/results/{run_id}/metrics_{model}.csv

artifacts/results/{run_id}/significance_tests.json

manifest.json per run

```
{
  "run_id": "a1b2c3d4",
  "config_hash": "...",
  "folds": [0,1,2,3,4],
  "models": ["hybrid_rgcn", "catboost", "..."],
  "artifacts": [{"path": "...", "sha256": "..."}]
}
```

Retention policy

- Keep best run + all baseline runs
- Delete intermediate epoch checkpoints; keep best.pt only

20. Result Generation

Module: scripts/09_generate_results.py

Outputs:

File	Content
paper/tables/table_main_metrics.csv	Mean±std AUPRC, F1, Recall, Precision, ROC-AUC
paper/tables/table_ablation.csv	A1–A9
paper/tables/table_significance.csv	p-values, Cohen's d, Holm-adjusted
paper/figures/fig_pr_curve.pdf	Mean PR curve Hybrid vs B4
paper/figures/fig_recall_at_k.pdf	$k \in \{50, 100, 506\}$
paper/figures/fig_confusion_hybrid.pdf	At 0.5 and val-opt threshold
paper/figures/fig_explanation_case.pdf	1 TP subgraph

evaluation/metrics.py functions

```
def compute_all_metrics(y_true, y_score, thresholds=(0.5,)) -> dict
def recall_at_k(y_true, y_score, k: int) -> float
def precision_at_k(y_true, y_score, k: int) -> float
def aggregate_cv_results(fold_metrics: list[dict]) -> dict # mean, std
```

21. Ablation Implementation

Module: `hgad_cms/evaluation/ablation.py`

```
ABLATION_REGISTRY = {
    "full": {"fusion": True, "shares_patient": True, "diagnosis": True, "encoder": "rgcn"},
    "rgcn_only": {"fusion": False, "alpha": 1.0, ...},
    "if_only": {"fusion": False, "alpha": 0.0, ...},
    "no_shares_patient": {"shares_patient": False},
    "no_diagnosis": {"diagnosis": False},
    "homogeneous_sage": {"encoder": "graphsage"},
    "unweighted_bce": {"pos_weight": 1.0},
}
```

CLI:

```
python scripts/08_run_ablations.py --config configs/default.yaml --ablations
full,rgcn_only,if_only,no_shares_patient
```

| **Unit tests** | `test_ablation_config_differs`, `test_ablation_rgcn_only_alpha_one` |

22. Statistical Significance Testing

Module: `hgad_cms/evaluation/statistics.py`

```
def paired_ttest(scores_a: np.ndarray, scores_b: np.ndarray) -> dict
def wilcoxon_test(scores_a, scores_b) -> dict
def holm_bonferroni(p_values: list[float]) -> list[float]
def cohens_d(diff: np.ndarray) -> float
def compare_hybrid_vs_baselines(hybrid_folds, baseline_folds_dict) -> pd.DataFrame
```

Input: fold-wise AUPRC vectors length 5

Output: `significance_tests.json`

| **Unit tests** | `test_holm_monotonic`, `test_paired_ttest_perfect_diff` |

23. Memory Optimization for RTX 3050 4GB

Strategy	Implementation location	Setting
Coarsen bene/phy/dx nodes	graph/coarsening.py	top-20K caps
Mixed precision (AMP)	training/trainer.py	memory.amp=true
Gradient accumulation	training/trainer.py	gradient_accumulation=2
Neighbor sampling	training/sampler.py	[15,10] not full neighbor
num_bases=4 in R-GCN	models/rgcn.py	relation decomposition
Store graph on CPU, batch to GPU	trainer.py	data.to("cpu"); batch .to(device)
Clear cache each epoch	trainer.py	torch.cuda.empty_cache()
Embedding dim 64 not 128	config	default 64
Explain on CPU if OOM	explain/gnn_explainer.py	fallback flag
CatBoost on CPU threads=4	baselines/catboost_model.py	avoid GPU contention

OOM recovery protocol

trainer.py pseudologic — implement as real code

```
OOM_RECOVERY_STEPS = [  
    {"batch_size": 256},  
    {"batch_size": 128, "num_neighbors": [10, 5]},  
    {"hidden_dim": 32},  
    {"model.primary": "graphsage"},  
]
```

Memory budget estimate (target)

Component	VRAM
R-GCN 2-layer, batch 512, dim 64	~2.5–3.5 GB
GNNExplainer one provider	~1–2 GB
Headroom	≥0.5 GB

24. Risk Mitigation Strategies

Risk	Detection	Mitigation	Owner
R-GCN OOM	RuntimeError: CUDA OOM in logs	OOM recovery steps → GraphSAGE	Engineer
Hybrid $\leq$ CatBoost	Week 3 metrics gate	Report ablations; emphasize recall@k + explainability; error analysis	Team
Label noise inflates metrics	High train/val gap	IF fusion; report AUPRC not accuracy	Engineer
Explainer unstable	fidelity < 0.5	Reduce hops to 1; fix seeds; homo projection	Engineer
IF on fraud in train	Code review	Assert fit mask excludes y=1	Reviewer
Test provider in train features	Unit test fail	Fold-scoped aggregate fit	Engineer
1-person bottleneck	Phase slip >2 days	Cut ablations to A1–A4; defer SHAP appendix	Lead
Non-reproducible metrics	Hash mismatch	Fixed seeds + manifest	Reviewer

Phase gates (hard stops)

Gate	When	Pass criterion
G1	End Phase 1	Validator passes; 558211 claims
G2	End Phase 2	Graph builds <8GB RAM; provider masks correct
G3	End Phase 3	CatBoost CV completes; AUPRC > 0.50
G4	End Phase 5	R-GCN val AUPRC $\geq$ CatBoost OR fallback approved
G5	End Phase 7	Hybrid beats R-GCN-only on $\geq 3/5$ folds (H2)
G6	End Phase 9	All tables + significance JSON generated

CLI Entry Points (scripts/)

Script	Command	Phase
01_preprocess.py	make preprocess	1
02_create_splits.py	make splits	1
03_build_graphs.py	make graphs	2
04_train_baselines.py	make baselines	3
05_train_graphsage.py	make sage	4
06_train_rgc.py	make rgcn	5
07_train_hybrid.py	make hybrid	6–7
08_run_ablations.py	make ablations	9
09_explain.py	make explain	8
10_evaluate.py	make evaluate	10
11_generate_paper_assets.py	make paper	10

Makefile targets

preprocess splits graphs baselines sage rgcn hybrid explain ablations evaluate paper all test lint

Unit Test Matrix (minimum 40 tests)

Module	Required tests
data.loader	3
data.merger	4
data.splits	4
features.*	5
graph.builder	6
graph.masks	3
models.rgc	3

Module	Required tests
models.fusion	4
anomaly.if	3
evaluation.metrics	4
evaluation.statistics	3
explain.*	3
integration smoke	2 (mini end-to-end)

Run: `pytest tests/ -v --cov=hgad_cms --cov-report=term-missing`

PHASED IMPLEMENTATION ROADMAP

Assumption: 1 implementer, ~20 working days (4 weeks), reviewers async.

Phase 1: Data Pipeline

Duration	3 days
Deliverables	data/* modules, 01_preprocess.py, 02_create_splits.py, processed parquets, data/splits/fold_{0-4}.json, manifest.json
Validation checks	Validator passes G1; fraud rate per fold $9.35\% \pm 1\%$; 558211 claims; splits provider-disjoint; pytest data tests green
Expected outputs	data/processed/*.parquet, data/splits/*.json, docs/reproduction.md §1

Phase 2: Graph Construction

Duration	3 days
Deliverables	graph/*, features/entity_features.py, 03_build_graphs.py, 5 fold graphs
Validation checks	G2 gate; provider count=5410 per fold; no NaN in provider.x; edge counts logged; leakage unit test passes

Expected outputs	artifacts/graphs/fold_k/hetero_data.pt, graph_stats.json
-------------------------	--

Phase 3: Baselines

Duration	3 days
Deliverables	baselines/*, features/provider_aggregates.py, features/centrality.py, 04_train_baseline.py
Validation checks	G3 gate; B1–B4 complete 5-fold CV; metrics CSV; CatBoost AUPRC logged
Expected outputs	artifacts/results/{run_id}/metrics_baselines.csv

Phase 4: GraphSAGE (Fallback Encoder)

Duration	2 days
Deliverables	models/graphsage.py, graph/projection.py, 05_train_graphsage.py
Validation checks	Forward pass on 4GB GPU; val AUPRC recorded; interface matches R-GCN
Expected outputs	metrics_graphsage.csv, models/.../graphsage_fold_k.pt

Phase 5: R-GCN

Duration	4 days
Deliverables	models/rgcn.py, training/*, 06_train_rgcn.py, B5 rgcn_only CV
Validation checks	G4 gate; trains without OOM; val AUPRC $\geq$ sage OR documented fallback; early stopping works
Expected outputs	metrics_rgcn_only.csv, models/.../rgcn_fold_k.pt, embeddings/.../provider_z.npy

Phase 6: Isolation Forest

Duration	1 day
Deliverables	anomaly/*, IF integration in hybrid pipeline, B6 if_only CV
Validation checks	IF fit only on non-fraud train; B6 metrics complete; score distribution logged
Expected outputs	models/.../if_fold_k.joblib, metrics_if_only.csv

Phase 7: Fusion

Duration	2 days
Deliverables	models/fusion.py, 07_train_hybrid.py, full hybrid CV, α per fold
Validation checks	G5 gate; H2/H3 ablations; α not in {0,1} for best fold; hybrid metrics CSV
Expected outputs	metrics_hybrid.csv, best_alpha.json, main PR curve data

Phase 8: Explainability

Duration	3 days
Deliverables	explain/*, 09_explain.py, 50 providers/fold explained
Validation checks	Mean fidelity ≥ 0.70 (H5); 3 case studies exported; FP explanations documented
Expected outputs	artifacts/explanations/fold_k/*.json, paper/figures/fig_explanation_case.pdf

Phase 9: Experiments (Ablations + Stats)

Duration	2 days
Deliverables	evaluation/ablation.py, 08_run_ablations.py, statistics.py, Holm-corrected tests
Validation checks	A1–A4 complete; paired tests Hybrid vs B4; effect sizes computed
Expected outputs	table_ablation.csv, significance_tests.json

Phase 10: Final Evaluation

Duration	2 days
Deliverables	10_evaluate.py, 11_generate_paper_assets.py, README reproduction guide, experiment manifest
Validation checks	All hypotheses H1–H7 addressed in results doc; reproducibility checklist §12 complete
Expected outputs	paper/tables/*, paper/figures/*, artifacts/results/{run_id}/experiment.json, final report for co-authors

Timeline Summary

Phase	Days	Cumulative
1 Data	3	3
2 Graph	3	6
3 Baselines	3	9
4 GraphSAGE	2	11
5 R-GCN	4	15
6 IF	1	16
7 Fusion	2	18
8 Explain	3	21
9 Experiments	2	23
10 Final	2	25

Buffer: 3 days for OOM fallback, re-runs, reviewer-requested fixes → fits **4-week** calendar.

First 48 Hours — Engineer Action List

1. Initialize repo skeleton + pyproject.toml + pre-commit
2. Place Kaggle CSVs in data/raw/
3. Implement constants.py, loader.py, merger.py, validator.py
4. Run 01_preprocess.py → confirm G1

5. Implement splits.py → write 5 fold JSONs
6. Implement synthetic 50-provider graph test fixture for downstream modules
7. Reviewers: verify fold disjointness + fraud rate per fold manually on JSON

This document is the **single source of truth** for implementation. All engineering decisions not listed here are out of scope unless they fix bugs, leakage, or OOM failures within the finalized architecture.